

PROMPTBREEDER: SELF-REFERENTIAL SELF-IMPROVEMENT VIA PROMPT EVOLUTION

Chrisantha Fernando, Dylan Banarse, Henryk Michalewski, Simon Osindero, Tim Rocktäschel

Google DeepMind

{chrisantha, dylski, henrykm, osindero, rocktaschel}@google.com

ABSTRACT

Popular prompt strategies like Chain-of-Thought Prompting can dramatically improve the reasoning abilities of Large Language Models (LLMs) in various domains. However, such hand-crafted prompt-strategies are often sub-optimal. In this paper, we present PROMPTBREEDER, a general-purpose self-referential self-improvement mechanism that evolves and adapts prompts for a given domain. Driven by an LLM, Promptbreeder mutates a population of task-prompts, evaluates them for fitness on a training set, and repeats this process over multiple generations to evolve task-prompts. Crucially, the mutation of these task-prompts is governed by mutation-prompts that the LLM generates and improves throughout evolution in a self-referential way. That is, Promptbreeder is not just improving task-prompts, but it is also improving the mutation-prompts that improve these task-prompts. Promptbreeder outperforms state-of-the-art prompt strategies such as Chain-of-Thought and Plan-and-Solve Prompting on commonly used arithmetic and commonsense reasoning benchmarks. Furthermore, Promptbreeder is able to evolve intricate task-prompts for the challenging problem of hate speech classification.

1 INTRODUCTION

Prompting is central to the downstream performance of foundation models. For example, different prompt strategies¹ can have a significant impact on a model’s reasoning abilities (Wei et al., 2022; Nye et al., 2021; Zhou et al., 2022; Wang et al., 2022; Zhou et al., 2023; Wang et al., 2023b), multi-modal processing abilities (Yang et al., 2023b; Wang et al., 2023d), or tool use abilities (Yao et al., 2022; Schick et al., 2023). Furthermore, prompting can improve model distillation (Wang et al., 2023c; Hsieh et al., 2023) and it can be used to simulate agentic behavior (Wang et al., 2023a; Park et al., 2023; Wu et al., 2023). However, these prompt strategies are manually engineered. Since the specific way a prompt is phrased can have a dramatic effect on its utility (Madaan & Yazdanbakhsh, 2022), it raises the question of whether prompt engineering can be automated. Automatic Prompt Engineer (APE, Zhou et al., 2023) attempts to address this by generating an initial distribution of prompts using another prompt that infers the problem from a number of input-output examples from the dataset. However, Zhou et al. found “diminishing returns to further selection rounds as the quality seems to stabilize after three rounds”, and consequently abandoned the use of an iterative APE. We propose a solution to the problem of diminishing returns via a diversity maintaining evolutionary algorithm for self-referential self-improvement of prompts for LLMs.

Schmidhuber (1990) notes that the “program of a neural network is its weight matrix”. Consequently, this “program” can be changed in a self-referential way by the neural network itself (Schmidhuber, 1993; Irie et al., 2022). Such a neural network that improves itself, as well as improving the way it improves itself, might be an important stepping stone towards open-ended self-referential self-improvement of AIs (Schmidhuber, 2003). However, self-improvement via self-referential weight matrices is costly as it requires additional parameters that modify all of the model’s

¹See Appendix A for definitions of terminology.

	Method	LLM	MultiArith*	SingleEq*	AddSub*	SVAMP*	SQA	CSQA	AQuA-RAT	GSM8K
Zero-shot	CoT	text-davinci-003	(83.8)	(88.1)	(85.3)	(69.9)	(63.8)	(65.2)	(38.9)	(56.4)
	PoT	text-davinci-003	(92.2)	(91.7)	(85.1)	(70.8)	—	—	(43.9)	(57.0)
	PS	text-davinci-003	(87.2)	(89.2)	(88.1)	(72.0)	—	—	(42.5)	(58.2)
	PS+	text-davinci-003	(91.8)	(94.7)	(92.2)	(75.7)	(65.4)	(71.9)	(46.0)	(59.3)
	PS	PaLM 2-L	97.7	90.6	72.4	83.8	50.0	77.9	40.2	59.0
	PS+	PaLM 2-L	92.5	94.7	74.4	86.3	50.1	73.3	39.4	60.5
	APE	PaLM 2-L	95.8	82.2	72.2	73.0	38.4	67.3	45.7	77.9
	OPRO	PaLM 2-L	—	—	—	—	—	—	—	80.2
	PB (ours)	PaLM 2-L	99.7	96.4	87.8	90.2	71.8	85.4	62.2	83.9
Few-	Manual-CoT	text-davinci-003	(93.6)	(93.5)	(91.6)	(80.3)	(71.2)	(78.3)	(48.4)	(58.4)
	Auto-CoT	text-davinci-003	(95.5)	(92.1)	(90.8)	(78.1)	—	—	(41.7)	(57.1)
	PB (ours)	PaLM 2-L	100.0	98.9	87.1	93.7	80.2	85.9	64.6	83.5

Table 1: Promptbreeder (PB) comparison to Chain-of-Thought (Manual-CoT, Wei et al., 2022), Zero-shot CoT (Kojima et al., 2022), Program-of-Thoughts (PoT, Chen et al., 2022), Auto-CoT (Zhang et al., 2023b), OPRO (Yang et al., 2023a), Automatic Prompt Engineer Zero-shot prompt (APE, Zhou et al., 2023), Plan-and-Solve with (PS+) and without the improved prompt (PS, Wang et al., 2023b) and using PaLM 2-L (Anil et al., 2023) as the underlying LLM (APE, PS_{PaLM 2-L}/PS+_{PaLM 2-L}). Best results in both the zero-shot and few-shot categories are highlighted in bold. Results in brackets are directly taken from the Plan-and-Solve paper which uses text-davinci-003 (Brown et al., 2020). For datasets with astericks (MultiArith*, SingleEq*, AddSub*, and SVAMP*), we randomly took half of the examples for training and report accuracy on the remaining test set. See Section 4 and Appendix I for details on the prompts and datasets.

parameters. Since behaviors and capabilities of LLMs are significantly influenced by the prompts that we provide to them, we can similarly think of prompts as the program of an LLM (Zhou et al., 2023). In this view, changing a prompt strategy such as the Scratchpad method (Nye et al., 2021) or Chain-of-Thought Prompting (Wei et al., 2022) corresponds to changing the “program” of the LLM. Taking this analogy further, we can use the LLM itself to change its prompts, as well as the way it changes these prompts, moving us towards a fully self-referential self-improving systems grounded in LLMs.

In this paper, we introduce PROMPTBREEDER (PB) for self-referential self-improvement of LLMs. Given a seed set of mutation-prompts (i.e. instructions to modify a task-prompt), thinking-styles (i.e. text descriptions of general cognitive heuristics), and a domain-specific problem description, PB generates variations of the task-prompts and mutation-prompts, exploiting the fact that LLMs can be prompted to act as mutation operators (Meyerson et al., 2023). Based on the fitness of the evolved task-prompts as measured on the training set, we select a subset of evolutionary units consisting of task-prompts and their associated mutation-prompt, to transmit to future generations. Over multiple generations of PB, we observe prompts adapting to the domain at hand. For example, in a mathematical domain, PB evolved the task-prompt "Show all your working. II. You should use the correct mathematical notation and vocabulary, where appropriate. III. You should write your answer in full sentences and in words. IV. You should use examples to illustrate your points and prove your answers. V. Your workings out should be neat and legible" on GSM8K (see Appendix J). On a wide range of commonly used benchmarks spanning commonsense reasoning, arithmetic, and ethics, we find that PB outperforms state-of-the-art methods like Chain-of-Thought (Wei et al., 2022) and Plan-and-Solve (Wang et al., 2023b) prompting. As PB does not require any parameter updates for self-referential self-improvement, we believe this approach points to an interesting future where larger and more capable LLMs could further amplify the gains of our approach.

In summary, this paper makes the following main contributions: (i) we introduce Promptbreeder, a self-referential self-improvement method for LLMs that evolves prompts for a domain at hand, as well as improves the way it is evolving these prompts, (ii) we report improvements over state-of-the-art prompt strategies on a wide range of commonly used arithmetic and commonsense reasoning benchmarks, and (iii) we investigate the various self-referential components of Promptbreeder and their contribution to our results.

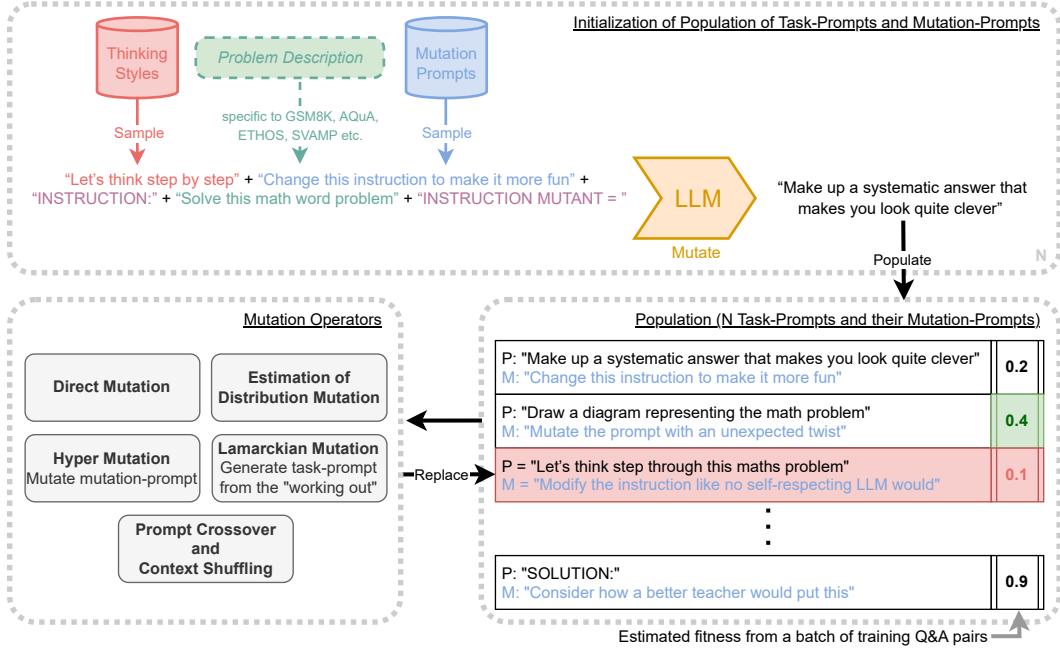


Figure 1: Overview of Promptbreeder. Given a problem description and an initial set of general “thinking-styles” and mutation-prompts, Promptbreeder generates a population of units of evolution, each unit consisting of typically two task-prompts and a mutation-prompt. We then run a standard binary tournament genetic algorithm (Harvey, 2011). To determine the fitness of a task-prompt we evaluate its performance on a random batch of training data. Over multiple generations, Promptbreeder subsequently mutates task-prompts as well as mutation-prompts using five different classes of mutation operators. The former leads to increasingly domain-adaptive task-prompts whereas the latter evolves increasingly useful mutation-prompts in a self-referential way.

2 RELATED WORK

Prompting an LLM in the right way is essential to its downstream performance (Moradi & Samwald, 2021; Madaan & Yazdanbakhsh, 2022; Zhou et al., 2023). Indeed, even the order in which prompts are presented can heavily influence LLM performance (Lu et al., 2022). A number of recent works have focused on devising better prompt strategies, or even automating such prompt engineering.

Prompting: Chain-of-Thought Prompting (CoT, Wei et al., 2022) is a popular prompt strategy which provides intermediate reasoning steps as few-shot prompts to an LLM, thereby significantly improving its arithmetic, commonsense, and symbolic reasoning abilities. Notably, the gains of CoT are more pronounced for stronger LLMs. This is intriguing, as it points to the possibility of increasingly capable (and potentially open-ended) self-improving mechanisms on top of adept LLMs—a hypothesis that Promptbreeder directly builds upon. Instead of few-shot CoT prompting, Kojima et al. (2022) demonstrate that LLMs can also be prompted zero-shot (e.g. "Let's think step by step") to produce their own chains of thoughts (Zero-shot CoT) that improve reasoning abilities. Self-Consistency (CoT-SC, Wang et al., 2022) extends CoT by sampling a diverse set of workings out and selecting the most consistent answer. Tree of Thoughts (ToT, Yao et al., 2023) generalizes CoT to multiple workings out that can be expanded or backtracked from. Graph of Thoughts (GoT, Besta et al., 2023) is a further generalization to arbitrary graph structures. Plan-and-Solve Prompting (PS, Wang et al., 2023b) encourages an LLM to first devise a plan to solve a problem before attempting to solve it. Similarly, Least-to-Most Prompting (Zhou et al., 2022) encourages an LLM to decompose a problem into subparts, and then to solve each part individually before synthesizing an answer. Self-Refine (Madaan et al., 2023) prompts an LLM to generate a response, to provide feedback on the response, and to finally refine the solution.

In contrast to gradient-free approaches above, Soft Prompting approaches (e.g., [Liu et al., 2021](#); [Qin & Eisner, 2021](#); [Lester et al., 2021](#)) directly fine-tune continuous prompt representations. [Huang et al. \(2022\)](#) use CoT and CoT-SC on an unlabelled dataset of questions, and subsequently fine-tune an LLM based on generated solutions. Similarly, [Zelikman et al. \(2022\)](#) uses CoT to generate rationales and fine-tunes the LLM based on those examples and rationales that yielded the correct answer. However, as argued by [Zhou et al. \(2023\)](#), any approach that updates all or a portion of LLM parameters will not scale as models get bigger and, moreover, will not work with the increasing number of LLMs hidden behind an API.

All of the prompt engineering approaches above are domain agnostic but hand designed. Central to our work is the hypothesis that we could do better by employing an automated self-improvement process that can adapt prompts to a domain at hand. Auto-CoT ([Zhang et al., 2023b](#)) and Automatic-CoT ([Shum et al., 2023](#)) automatically find reasoning chains for Few-Shot CoT. Automatic Prompt Engineer (APE, [Zhou et al., 2023](#)) uses one generator-prompt to generate prompt candidates, and another mutation-prompt to mutate them. In contrast to APE, our work performs compositional task-specific initialization of mutation-prompts, subsequent online mutation of mutation-prompts, uses special mutation operators that take into account the whole population and elite history, and uses diversity-maintenance methods—all of which help avoid the problem of diminishing returns and diversity loss suffered by APE.

Concurrently to our work, [Yang et al. \(2023a\)](#) developed Optimization by PROMpting (OPRO), a prompt optimization method that varies prompts using a single complex mutation prompt, and evaluates newly generated prompts on a small fixed training set of problems. In contrast, Promptbreeder autonomously evolves multiple LLM generated mutation-prompts as well as task-prompts, and evaluates fitness on random subsets from the whole training set during evolution. At the time of its release, OPRO achieved a score of 80.2% via the optimized zero-shot prompt "Take a deep breath and work on this problem step-by-step" on GSM8K. Promptbreeder surpasses this with 83.9% in the zero-shot setting with the unintuitively simple prompt "SOLUTION"—further evidence for the sensitivity of LLMs to prompts and the importance on finding effective prompts automatically. Also concurrently to our work, [Guo et al. \(2023\)](#) developed EvoPrompt, which uses a fixed mutation (and crossover) prompt, as well as a prompt that asks for a mutant of the difference between two parent prompts, to produce offspring prompts. EvoPrompt is initialized with a whole population of initial hand-designed task tailored prompts rather than a single problem description as we do. In contrast to the two approaches above, Promptbreeder uses LLMs to self-referentially improve mutation-prompts, and it is able to evolve contexts as well.

Self-Referential Self-Improvement: Developing an open-ended system that can improve itself as well as improving the way it is improving itself ([Schmidhuber, 1993; 2003](#)) is a long-standing open problem in AI research. [Schmidhuber \(1993\)](#) introduced an “introspective” neural network with a self-referential weight matrix that can modify its own weights and, thus, also modify those weights that are governing how its own weights are modified. Recently, [Irie et al. \(2022\)](#) proposed a more scalable self-referential weight matrix taking inspiration from fast weight programmers ([Schmidhuber, 1992](#)). [Kirsch & Schmidhuber \(2022\)](#) propose a self-referential meta-learning approach, combining self-referential weight matrices with ideas from Gödel Machines ([Schmidhuber, 2003](#)), i.e., to allocate more computational resources to better performing solutions. However, since these approaches directly modify parameters of a model, it is unclear how to scale them to the increasing number of parameters in modern LLMs. In contrast, for Promptbreeder the substrate of self-referential self-improvement is natural language, avoiding costly parameter updates altogether.

Open-Endedness and LLMs: Promptbreeder makes use of the observation by [Lehman et al. \(2022\)](#), [Meyerson et al. \(2023\)](#) and [Chen et al. \(2023\)](#) that LLMs are effective at generating mutations from examples. In addition, LLMs encode human notions of interestingness and can be used to automatically quantify novelty ([Zhang et al., 2023a](#)). Promptbreeder is related to Picbreeder ([Secretan et al., 2008](#)), an open-ended human-in-the-loop system that evolves increasingly interesting images. While Picbreeder explores the space of images, Promptbreeder explores the space of prompts and does so without humans in the loop. As Promptbreeder is proposing mutated prompts to itself, it is an example of a system transitioning from “learning from data” to “learning what data to learn from” ([Jiang et al., 2022](#)).

3 PROMPTBREEDER

We introduce Promptbreeder, a prompt evolution system that can automatically explore prompts for a given domain and that is able to find task-prompts that improve an LLM’s ability to derive answers to questions in that domain. Promptbreeder is general purpose in that the same system is able to adapt to many different domains.

Promptbreeder makes use of the observation that LLMs can be used to generate variations of input text (Lehman et al., 2022; Meyerson et al., 2023; Chen et al., 2023). Figure 1 gives an overview of our method. We are interested in evolving task-prompts. A task-prompt P is a string used to condition the context of an LLM in advance of some further input Q , intended to ensure a better response than if Q had been presented in the absence of P . To evaluate the fitness of each evolved task-prompt, we sample a batch of 100 Q&A pairs from the entire training set of the domain at hand.²

Promptbreeder generates task-prompts according to an evolutionary algorithm. The mutation operator for this algorithm is itself an LLM, conditioned on a mutation-prompt M . That is, a mutated task prompt P' is defined by $P' = \text{LLM}(M + P)$ where ‘+’ corresponds to string concatenation. A variety of such mutation-prompts are described in Section 3.2.

Promptbreeder’s main self-referential mechanism stems from applying the evolutionary algorithm not just to task-prompts but also to mutation-prompts. The mutation operator for this meta-level algorithm is again an LLM, now conditioned on a hyper-mutation prompt H . That is, we obtain a mutated mutation-prompt M' via $M' = \text{LLM}(H + M)$.

Given a set of “thinking styles” $\mathcal{T}$ and a set of initial mutation-prompts $\mathcal{M}$, as well as a domain-specific problem description D , Promptbreeder initializes a population of mutated task-prompts (see Section 3.1). To clarify, a unit of evolution consists of a set of task-prompts, a mutation-prompt and in the few-shot case, a set of correct workings out (i.e. step-by-step or “chains-of-thought” reasoning steps that led to the correct answer). This means task-prompts and mutation-prompts are in 1:1 correspondence. To evolve this population, we employ a binary tournament genetic algorithm framework (Harvey, 2011): we sample two individuals from the population, we take the individual with the higher fitness, mutate it (see next section) and overwrite the loser with the mutated copy of the winner.

3.1 PROMPTBREEDER INITIALIZATION

To give a concrete example, consider the initialization steps used to produce the task-prompts and mutation-prompts for GSM8K (a ‘grade school maths’ word problem dataset). The problem description is "Solve the math word problem, giving your answer as an arabic numeral". Because Plan-and-Solve (Wang et al., 2023b) uses two task-prompts we also evolve two task-prompts (plus a mutation-prompt) per unit of evolution. In order to promote diversity in the initial prompts, we generate the initial task-prompts by concatenating (for each task-prompt) a randomly drawn ‘mutation-prompt’ (e.g. "Make a variant of the prompt.") and a randomly drawn ‘thinking-style’ (e.g. "Let’s think step by step") to the problem description, and provide that to the LLM to produce a continuation, resulting in an initial task-prompt. We do this twice to produce the two initial task-prompts per unit. Both the mutation-prompt and the thinking-style are randomly sampled from an initial set of mutation-prompts and a set of thinking-styles (see Appendices C, D and G for the full sets). The mutation-prompt is added to the unit of evolution and so is associated with its specific task-prompt throughout the evolutionary run.

For the example above, the complete input string to the LLM to make an initial task-prompt could be "Make a variant of the prompt. Let’s think step by step. INSTRUCTION: Solve the math word problem, giving your answer as an arabic numeral. INSTRUCTION MUTANT:". Note how the control strings "INSTRUCTION" and "INSTRUCTION MUTANT" are added to encourage an appropriate continuation. Table 4 in Appendix E shows examples of the initial prompts generated in this way.

²Our prompt strategy sequentially applies two task-prompts. The first task-prompt + question produces a continuation. The continuation + second task-prompt produces the final answer.

3.2 MUTATION OPERATORS

As shown in Figure 1, there are nine operators falling into five broad classes which drive the exploration of prompt strategies. For each replication event only one of nine mutation operators is applied (we sample with uniform probability over the nine operators to decide which mutation operator to apply). The rationale for using this diverse set of operators is to enable the LLM to explore a large space of cognitive methods of linguistic self-questioning, by repeatedly changing the framing of the problem as well as retrieving mental models expressed in natural language that can help tackle a given reasoning challenge. Investigations from insight learning strongly suggest that diverse representational re-description is key to problem solving (Öllinger & Knoblich, 2009)—a principle that we attempt to recreate via self-referential self-improvement with natural language as the substrate. Figure 2 illustrates in what way Promptbreeder is self-referential (see Appendix F for a more detailed explanation).

3.2.1 DIRECT MUTATION

The simplest class of mutation operators directly generate a new task-prompt P' from either one existing task-prompt P (first-order prompt generation) or from a general prompt that encourages free-form generation of new task-prompts—i.e. not using an existing parent, thus zero-order prompt generation.

Zero-order Prompt Generation: We generate a new task-prompt by concatenating the problem description D (e.g. "Solve the math word problem, giving your answer as an arabic numeral") with the prompt "A list of 100 hints:", which invites the LLM to come up with a new hint that could help solve a problem in the given problem domain. We extract the first generated hint as the new task-prompt. Crucially, this new task-prompt does not depend on any previously found task-prompt. Instead, it is re-generated from the problem description each time. Our rationale for including this zero-order operator is that where prompt evolution diverges, this operator allows us to generate new task-prompts closely related to the original problem description, similar to uniform re-sampling in automated curriculum learning approaches (Jiang et al., 2021b;a; Park et al., 2023; Parker-Holder et al., 2022).

First-order Prompt Generation: We concatenate the mutation-prompt (red), to the parent task-prompt (blue), and pass it to the LLM to produce the mutated task-prompt. For example "Say that instruction again in another way. DON'T use any of the words in the original instruction there's a good chap. INSTRUCTION: Solve the math word problem, giving your answer as an arabic numeral. INSTRUCTION MUTANT: ". This procedure is identical to the initialization method, except that a randomly sampled thinking-style string is not used. First-order prompt generation is Promptbreeder's standard asexual mutation operator, and it is the core of every genetic algorithm—taking one parental genotype (task-prompt) and applying the mutation to it (in this case influenced by the mutation-prompt).

3.2.2 ESTIMATION OF DISTRIBUTION MUTATION

The next class of mutation operators condition not just on zero or one parent, but instead on a set of parents. As such, they may be more expressive by considering patterns in the population.

Estimation of Distribution (EDA) Mutation: Inspired by Hauschild & Pelikan (2011), we provide a filtered and numbered list of the current population of task-prompts to the LLM and ask it to continue this list with new task-prompts. We filter the population of prompts on the basis of BERT (Devlin et al., 2019) embedding cosine similarities between each other—an individual is not included in the list if it is more than 0.95 similar to any other entry in the list, thus encouraging diversity (cf. quality-diversity methods (Lehman & Stanley, 2011b;a; Mouret & Clune, 2015)). The prompts are listed in random order and we do not give the LLM access to the fitness values of individuals in the population—we found in preliminary experiments that the LLM did not understand these fitness values³ and resorted to generating copies of entries in the list.

³This is contrary to recent findings by Mirchandani et al. (2023). We leave it for future work to revisit whether LLMs can interpret fitness values for improved prompt evolution.

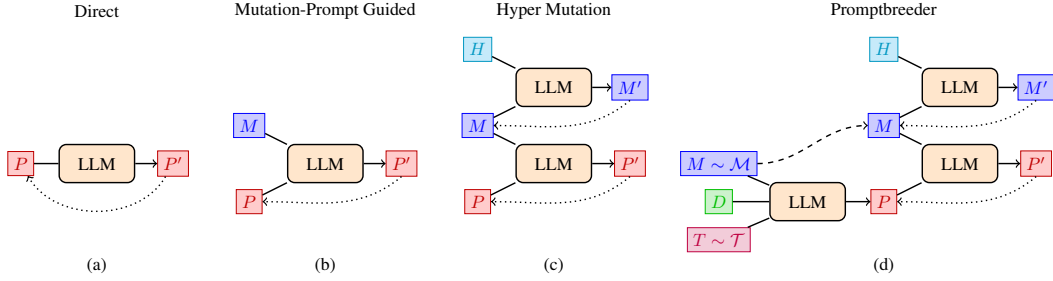


Figure 2: Overview of multiple variants of self-referential prompt evolution. In (a), the LLM is directly used to generate variations P' of a prompt strategy P (cf. Meyerson et al., 2023). Using a mutation prompt M , we can explicitly prompt an LLM to produce variations (b). By using a hyper mutation prompt H , we can also evolve the mutation prompt itself, turning the system into a self-referential one (c). Promptbreeder (d) improves the diversity of evolved prompts and mutation prompts by generating an initial population of prompt strategies from a set of seed thinking-styles $\mathcal{T}$, mutation-prompts $\mathcal{M}$, as well as a high level description D of the problem domain.

EDA Rank and Index Mutation: This is a variant of the above in which task-prompts are listed in fitness order. Preliminary experiments showed that the LLM is more likely to generate entries that are similar to the elements appearing later in the list. This is in line with similar findings of recency effects in LLMs (Liu et al., 2023). Therefore, after filtering in the same way as before, we ordered the task-prompts in the population by ascending order of fitness. The top of the list is prefixed by the following prompt: "INSTRUCTION: " + <<mutation-prompt>> + "\n A List of Responses in descending order of score." + <<last index + 1>> + "is the best response. It resembles" + << last index>> + "more than it does (1)". Note that we have ‘lied’ to the LLM by telling it that the order is descending. This is because otherwise it is too biased towards producing a new entry that is too similar to the final entry. The contradiction between the ascending ordering and the statement that it is a descending ordering appears to improve the diversity of sampling. The rationale for this operator is again to represent the current distribution in such a way that high fitness and yet diverse extrapolations are suggested by the LLM.

Lineage Based Mutation: For each unit of evolution, we store a history of the individuals in its lineage that were the best in the population, i.e., a historical chronological list of elites. This list is provided to the LLM in chronological order (not filtered by diversity), with the heading "GENOTYPES FOUND IN ASCENDING ORDER OF QUALITY" to produce a novel prompt as continuation. The rationale for this operator is that we expect the signal of improving genotype prompts may be stronger than the signal from prompts in the current population since they provide a gradient of bad to good prompts that could be followed (assuming this signal can be used by the LLM).

3.2.3 HYPERMUTATION: MUTATION OF MUTATION-PROMPTS

While the mutation operators above might already explore diverse task-prompts, a self-improving system should ideally also improve the way it is improving itself in a self-referential way. Our third class of mutation operators includes hyper-mutation operators concerned with the evolution of evolvability (Dawkins, 2003; Pigliucci, 2008; Payne & Wagner, 2019; Gajewski et al., 2019)—those which modify the search/exploration process rather than the task reward obtaining process directly.⁴

Zero-order Hyper-Mutation: We concatenate the original problem description to a randomly sampled thinking-style, and feed it to the LLM to generate a new mutation-prompt. The resulting mutation-prompt is applied to a task-prompt to make a variant of the task-prompt as in First-order Prompt Generation (see Section 3.2.1). Note that this zero-order meta-mutation operator is identical to that used during initialization. The rationale for this operator is to generate mutation operators in a way similar to initialization, while also bringing in knowledge from the set of thinking styles.

⁴This is similar to population based training (Jaderberg et al., 2017a)—instead of applying it to hyperparameters such as learning rates, it applies to the mutation-prompts of Promptbreeder.

First-order Hyper-Mutation: We concatenate the hyper-mutation-prompt "Please summarize and improve the following instruction:" to a mutation-prompt so that the LLM generates a new mutation-prompt. This newly generated mutation-prompt is then applied to the task-prompt of that unit (see First-Order Prompt Generation in Section 3.2.1). In this way, we can evaluate the influence of the hyper-mutation via its newly generated mutation-prompt on the quality of the evolved downstream task-prompt at once.

3.2.4 LAMARCKIAN MUTATION

For this class of mutation operators we mimic a Lamarckian process. We want to use a successful phenotype (i.e. the concrete working out used to produce correct answers induced by an evolved task-prompt) to generate a new genotype (i.e. a mutant task-prompt). Several processes of this form have appeared in the literature of LLMs, e.g. STaR (Zelikman et al., 2022), APO (Pryzant et al., 2023), and APE (Zhou et al., 2023).

Working Out to Task-Prompt: This is a ‘Lamarckian’ mutation operator similar to instruction induction in APE. We give an LLM a previously generated working out that led to a correct answer via the following prompt: "I gave a friend an instruction and some advice. Here are the correct examples of his workings out + <<correct working out>> + The instruction was:". This is effectively reverse-engineering the task-prompt from a given working out. An effective example of this is shown in Appendix H. This kind of operator is critical when the problem description is absent, insufficient, or misleading.

3.2.5 PROMPT CROSSOVER AND CONTEXT SHUFFLING

Our last class of mutation operators are crossover operators and operators for shuffling the few-shot context examples present in the units of evolution.

Prompt Crossover: After a mutation operator is applied, with 10% chance a task-prompt is replaced with a randomly chosen task-prompt from another member of the population. This member is chosen according to fitness proportionate selection. Crossover is not applied to mutation-prompts, only to the task-prompts.

Context Shuffling: Promptbreeder can simultaneously evolve the task-prompts, mutation-prompts and the set of correct workings out known as the few-shot context. To achieve the later, we fill up a few-shot context with only workings out that led to correct answers. During evaluation we provide this few shot-context before the task-prompt, providing guidance as to the form of the working out that is desired. If the few-shot context list is full, a single randomly sampled new correct working out replaces an existing working out from the list after fitness evaluation of a unit on a new set of questions. In addition, with a 10% chance we resample the whole context list with probability inverse to the maximum context list length.

4 EXPERIMENTS

We used a population size of 50 units, evolved for typically 20-30 generations, where a generation involves forming random pairs of all individuals in the population and competing them against each other. To evaluate Promptbreeder, we use the datasets from state-of-the-art prompt strategies such as Plan-and-Solve, spanning *arithmetic reasoning* with GSM8K (Cobbe et al., 2021), SVAMP (Patel et al., 2021), MultiArith (Roy & Roth, 2016), AddSub (Hosseini et al., 2014), AQUA-RAT (Ling et al., 2017), and SingleEq (Koncel-Kedziorski et al., 2015), *commonsense reasoning* with CommonsenseQA (CSQA, Talmor et al., 2019) and StrategyQA (SQA, Geva et al., 2021), *instruction induction* tasks from (Honovich et al., 2023), and *hate speech classification* on the ETHOS dataset (Mollas et al., 2022). See Appendix I for details.

5 RESULTS AND DISCUSSION

We present results of Promptbreeder (PB) in comparison to state-of-the-art prompt strategies on a range of commonly used reasoning benchmarks in Table 1. PB outperforms PS+, the best Plan-and-Solve (Wang et al., 2023b) prompting technique. Note that the performance of PS+ is improved

by using PaLM 2-L (Anil et al., 2023) as the underlying LLM (**PS+PaLM 2-L**) on all datasets except ADDSUB compared to text-davinci-003 results in the original paper. On all other datasets, zero-shot PB accuracy is higher than PS+, with further improvement in the few-shot case when examples of discovered solutions are included with the prompts. In Table 6 in Appendix J, we show the best evolved zero-shot prompts. The best few-shot candidates are shown in Appendix J.5 onwards. Appendix K shows few-shot results and their controls on the Instruction Induction tasks from the APE paper. To investigate the ability of Promptbreeder to evolve complex domain-specific prompts for a downstream task, we applied it to the ETHOS Hate Speech Classification problem (Mollas et al., 2022). Promptbreeder was able to evolve a prompt strategy consisting of two sequentially applied relatively long prompts (see Appendix J.1) that scored 89% on ETHOS—an improvement over the hand-designed prompt "Determine whether a text contains hate speech" which scores only 80%. This demonstrates that Promptbreeder is capable of intricate domain-adaptation to a task at hand. Appendix B shows a typical evolutionary run and the prompts evolved, showing that unlike iterative APE, fitness continues to increase throughout the run.

We analysed the best mutation-prompts used during a run for GSM8K. Table 7 in Appendix J.3 shows the best evolved mutation prompts according to their scores (the proportion of times that when the mutation-prompt was applied to a task-prompt in an unit, a better task-prompt was produced). Table 8 in Appendix J.4 shows in descending order, the percentage of times that the different kinds of mutation operators resulted in an improvement when applied to a task-prompt in the population. It demonstrates that all mutation operators are important for Promptbreeder to work, including hypermutation operators which lead to self-referential self-improvement.

We measured the impact of self-referential operators on all the maths datasets and the ETHOS dataset. Details of the ablation process and its results can be found in Appendix L. Removing any self-referential operator is harmful under nearly all circumstances, the greatest benefit being the initial re-description of task-prompts upon initialization. We only found one mutation operator to be harmful for one specific task: drawing randomly from the set of mutation-prompts upon initialization hurts performance on GSM8K.

6 CONCLUSION AND FUTURE WORK

We introduced PROMPTBREEDER (PB), a self-referential self-improving system that can automatically evolve effective domain-specific prompts for a domain at hand. PB is self-referential in that it not only evolves task-prompts, but it also evolves mutation-prompts that govern the way PB modifies task-prompts. Thus, it is not only improving prompts but it also improves the way it is improving prompts.

Going forward, it could be interesting to use the LLM itself to assess and promote the diversity of generated prompts (see Zhang et al., 2023a), or to use it to determine the fitness of a whole “thought process”, e.g. an N-prompt strategy where prompts are conditionally applied rather than unconditionally applied as in Promptbreeder. For example, a more complex “thought process” is to use PB in self-play mode to evolve pre-prompts for LLM-based policies that compete with each other, i.e., in a competitive Socratic⁵ dialog.

PB remains limited compared to the open-endedness of human thought processes. First, the topology of prompting remains fixed (see Figure 2)—we only adapt the prompt content not the prompting algorithm itself. One interpretation of thought is that it is a reconfigurable open-ended self-prompting process. If so, how does one develop complex thought strategies? Clearly it is necessary to generate and evaluate them, and whilst a simple evolutionary process provides one framework in which a thought strategy could be evolved, our actual human experience suggests multiple overlapping hierarchical selective processes at play. Moreover, in addition to language, human thought involves intonation, imagery, etc., in a multimodal system.

We believe PB points to an exciting future where increasingly open-ended self-referential self-improvement systems can directly use language as the substrate for improvement instead of relying on any parameter updates. This is intriguing, as this approach will likely continue to scale with ever larger and more capable LLMs in the future.

⁵<https://princeton-nlp.github.io/SocraticAI/>